

Refatoração de Testes e Detecção de Test Smells

Disciplina: Testes de Software
Trabalho: Refatoração de Testes e Detecção de Test Smells
Aluno: Gabriel Nogueira Vieira Resende
Matrícula: 854405
Repositório:

1. Análise de Smells

Analisei manualmente o arquivo `userService.smelly.test.js` e identifiquei três Test Smells principais.

1.1 Lógica Condicional no Teste (Conditional Test Logic)

Onde: teste deve desativar usuários se eles não forem administradores.

O teste percorre um array com um `for` e usa um `if/else` para decidir qual `expect` executar. Usuário comum e administrador passam por ramos diferentes dentro do mesmo teste.

Por que é um mau cheiro: o teste deve ser a especificação mais simples possível do comportamento. Com `if` e `for`, o leitor precisa "executar o código de cabeça" para saber o que está sendo verificado. Além disso, o ramo testado depende de `user.isAdmin`, um dado do próprio objeto criado, e não de uma expectativa fixa.

Risco: se o ramo esperado não for executado, por exemplo por causa de um dado errado, nenhum `expect` roda e o teste passa sem verificar nada. O teste também não confirma que o administrador continua `ativo` depois da tentativa de desativação.

1.2 Teste que passa sem verificar (Exception Handling com try/catch)

Onde: teste deve falhar ao criar usuário menor de idade.

```
try {
  userService.createUser('Menor', 'menor@email.com', 17);
} catch (e) {
  expect(e.message).toBe('0 usuário deve ser maior de idade.');
```

Por que é um mau cheiro: o `expect` só roda se uma exceção for lançada. Se a exceção não ocorrer, o teste termina sem nenhuma asserção e é considerado aprovado.

Risco: falso positivo. Se a validação de idade for removida do `createUser`, o bug passa despercebido e o teste continua verde. É exatamente o tipo de teste que deixa mutantes sobreviverem no Teste de Mutação.

1.3 Teste Frágil (Fragile Test)

Onde: teste deve gerar um relatório de usuários formatado.

O teste compara o relatório com a string exata `ID: ${id}, Nome: Alice, Status: ativo\n` e usa `startsWith(...).toBe(true)`.

Por que é um mau cheiro: o teste está acoplado ao formato do texto (vírgulas, espaços, quebra de linha), e não ao comportamento, que é listar os usuários com nome e status. Qualquer mudança cosmética no relatório quebra o teste, embora a funcionalidade continue correta. O comentário no próprio código reconhece que o formato "pode mudar no futuro".

Risco: o custo de manutenção aumenta e a equipe passa a ignorar testes vermelhos. Além disso, `toBe(true)` produz a mensagem de falha "esperava true, recebeu false", que não diz o que deu errado.

1.4 Outros smells encontrados

Smell	Onde	Observação
Eager Test	deve criar e buscar um usuário corretamente	Testa <code>createUser</code> e <code>getUserById</code> no mesmo teste, com dois "Acts".
Assertion Roulette	testes 1, 2 e 3	Várias asserções sem mensagem, então não fica claro qual falhou.
Teste ignorado	<code>test.skip(...)</code>	Teste vazio com <code>TODO</code> . Cobre justamente o relatório vazio, que ficou sem verificação.
Magic Numbers	<code>30</code> , <code>40</code> , <code>17</code>	Valores soltos, sem nome que explique a intenção.
Lacunas de cobertura	—	Nenhum teste para campos obrigatórios, idade 18, id inexistente em <code>getUserById</code> e em <code>deactivateUser</code> .

2. Processo de Refatoração

Escolhi o teste mais problemático: o da desativação de usuários, que mistura `for`, `if/else` e `expect` condicional.

Antes (`userService.smelly.test.js`)

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

Depois (userService.clean.test.js)

```
describe('deactivateUser', () => {
  test('retorna true ao desativar um usuário comum', () => {
    // Arrange
    const usuarioComum = criarUsuarioComum(userService);

    // Act
    const resultado = userService.deactivateUser(usuarioComum.id);

    // Assert
    expect(resultado).toBe(true);
  });

  test('altera o status do usuário comum para inativo', () => {
    // Arrange
    const usuarioComum = criarUsuarioComum(userService);

    // Act
    userService.deactivateUser(usuarioComum.id);

    // Assert
    expect(userService.getUserById(usuarioComum.id).status).toBe('inativo');
  });

  test('retorna false ao tentar desativar um administrador', () => {
    // Arrange
    const usuarioAdmin = criarUsuarioComum(userService, { isAdmin: true });

    // Act
    const resultado = userService.deactivateUser(usuarioAdmin.id);

    // Assert
    expect(resultado).toBe(false);
  });

  test('mantém o administrador ativo após tentativa de desativação', () => {
    // Arrange
    const usuarioAdmin = criarUsuarioComum(userService, { isAdmin: true });

    // Act
    userService.deactivateUser(usuarioAdmin.id);

    // Assert
    expect(userService.getUserById(usuarioAdmin.id).status).toBe('ativo');
  });
});
```

Decisões tomadas

1. **Remoção do for e do if/else**. Cada cenário (comum e administrador) virou seu próprio teste, com um único caminho de execução. Assim os `expect` sempre rodam, e o smell de lógica condicional desaparece.
2. **Padrão AAA**. Cada teste tem as seções Arrange, Act e Assert, separadas e comentadas.
3. **Um comportamento por teste**. O retorno do método e a mudança de status viraram testes distintos. Quando um falha, o nome do teste já indica o problema.
4. **Nova verificação**. Passou a existir um teste que garante que o administrador continua `ativo`, lacuna do teste original.
5. **Helper `criarUsuarioComum`**. Ele centraliza a criação de usuários com valores padrão e permite sobrescrever só o que importa para o cenário (`{ isAdmin: true }`). Isso reduz a repetição e deixa clara a diferença entre os cenários.

Os outros smells foram corrigidos no mesmo arquivo:

- **try/catch**: trocado por `expect(() => ...).toThrow('O usuário deve ser maior de idade.')`. Agora o teste falha se a exceção não for lançada.
- **Teste frágil**: o relatório é verificado por expressões regulares que exigem id, nome e status, sem depender de vírgulas, espaços ou `\n`.

- **Eager Test e Assertion Roulette:** os testes foram divididos e, em geral, há uma asserção por teste.
- **Teste ignorado:** o caso do relatório vazio foi implementado.
- **Magic Numbers:** a constante `IDADE_MINIMA` substitui o `17` solto e permite testar o limite exato (18).
- **Cobertura:** foram adicionados testes para campos obrigatórios, id inexistente e idade limite.

3. Relatório da Ferramenta (ESLint)

Comando executado na primeira análise, depois de instalar `eslint@8.57.1` e `eslint-plugin-jest@28.11.0` e criar o `.eslintrc.json` do roteiro:

```
npx eslint test src
```

Saída da primeira execução no arquivo `userService.smelly.test.js` (o `clean.test.js` ainda era uma cópia idêntica e gerou os mesmos 6 problemas):

```
/Users/gabriel.nogueira/Desktop/personal/uni/test-smelly/test/userService.smelly.test.js
45:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
47:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
50:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
74:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
78:3  warning  Disabled test                          jest/no-disabled-tests
78:3  warning  Test has no assertions                  jest/expect-expect
```

*** 6 problems (4 errors, 2 warnings)**

Comparação com a análise manual

Smell	Detectado pelo ESLint?	Regra
Lógica condicional (if/else no for)	Sim	jest/no-conditional-expect (linhas 45, 47, 50)
try/catch com expect no catch	Sim	jest/no-conditional-expect (linha 74)
Teste ignorado e vazio	Sim	jest/no-disabled-tests, jest/expect-expect
Teste frágil (formato exato do relatório)	Não	Exige julgamento humano
Eager Test	Não	Exige julgamento humano
Assertion Roulette, Magic Numbers	Não	Não cobertos pelas regras ativas

A ferramenta automatizou a detecção dos smells estruturais, que têm um padrão sintático claro: `expect` dentro de condicionais ou blocos `catch` e testes desabilitados ou sem asserções. Ela aponta arquivo, linha e regra em segundos e pode rodar no CI. Smells que dependem de semântica, como fragilidade e Eager Test, só a análise manual encontrou. As duas abordagens se complementam.

Validação final

- `npx eslint test/userService.clean.test.js --max-warnings 0` terminou com código 0, sem erros nem avisos.
- `npm test`: 2 suítes aprovadas, 24 testes passando e 1 ignorado (o `test.skip` do arquivo `smelly`, que não deve ser alterado).

4. Conclusão

Testes limpos funcionam como documentação executável. Quando cada teste segue AAA, verifica um único comportamento e tem um nome descritivo, uma falha indica rapidamente o que quebrou. Testes com lógica condicional, `try/catch` sem garantia de falha ou acoplados a formatos exatos geram falsos positivos, falsos negativos e custo de manutenção, e por isso deixam de proteger o código.

A análise estática complementa a revisão humana. O ESLint com `eslint-plugin-jest` impede que smells estruturais entrem no repositório, de forma rápida, barata e repetível, e pode bloquear um pull request. Ele não substitui o julgamento do desenvolvedor sobre fragilidade e escopo dos testes, mas libera tempo de revisão para o que exige análise humana.

Em conjunto, essas práticas tornam a suíte de testes confiável e fácil de evoluir, o que sustenta a qualidade do projeto ao longo do tempo.